# CueClub

## A Cue Sports Club & Tournament Management System

DBMS Course Project (CS F212 Database Systems) – Mid-Semester Submission

Group-18 · BITS Pilani, Hyderabad Campus · 11 October 2026

**Priyansh Thadhani** 2024A3PS0356H
**Goru Nikhil** 2024A3PS0357H
**Ananya Avunooru** 2024A3PS0367H
**Manya Bhateja** 2024A3PS0378H
**Ujjwal Chandak** 2024A3PS0379H

**GitHub:** `github.com/StuCCAn/DBMS-Project`

## 1) Problem Statement and Scope

A cue sports club that also fields a competitive team is run through many small, interdependent activities: table bookings, tournament brackets, customer credit (udhar), equipment and consumables, and the development of individual players. These are usually kept in paper registers and in the manager's memory, so bookings overlap, balances are disputed, brackets are lost and stock-outs are noticed late. CueClub Manager consolidates them in one relational database with enforced integrity, behind a web application, so that the manager is no longer the point of coordination.

**Intended users and key functionality:** table reservation with hourly or per-frame billing; per-member credit ledger; tournaments with seeding, bracket progression and frame-by-frame scoring; ranking, head-to-head and win-streak reports; skill assessments for coaching; inventory with reorder alerts and servicing history. **Out of scope:** payment gateways, live streaming, multi-club franchises and inter-club fixtures against outside players.

| User | Main functionality |
|---|---|
| Club manager (admin) | Full operational overview; manage inventory and servicing; approve credit limits; schedule tournaments; dispute resolution. |
| Staff / Scorer | Bookings, hourly and per-frame billing; manage udhar credit ledger; record frame scores and advance brackets. |
| Club member | Book tables; view outstanding ledger balance and discount status; track individual rankings and tournament history. |
| Team member & Captain | Team selection tracking; recorded skill assessments; participate in seeded tournament brackets. |
| Casual player | Walk-in table reservations paid upfront; view game availability (no credit facility). |

**Table 1:** Intended users and main functionality.

## 2) Assumptions and Business Rules

- **R1** A table cannot be booked for overlapping times; bookings are checked against active intervals and do not cross midnight.
- **R2** Only club members may hold credit (udhar), and casual players must pay upfront at the time of reservation.
- **R3** A member's outstanding balance may not exceed the assigned credit limit, and any settlement payment may not exceed the balance.
- **R4** A player registers once per tournament and assigned seeds are unique within that tournament. Player slots for later rounds stay empty until feeder matches complete.
- **R5** A match winner is strictly one of its two players, and the player winning more frames wins (best-of-N, no drawn frames permitted).
- **R6** Completing a match atomically records frame scores, advances the winner to the next round bracket, and updates leaderboard ranking points.
- **R7** Competitive team roles require an active, non-expired club membership.
- **R8** Inventory items falling below their specified reorder level are automatically flagged for restocking.
- **R9** Amounts are in INR; users with active historical records are deactivated rather than physically deleted.

## 3) ER/EER Design

The database design models club operations, user specialization hierarchies, and competitive fixtures using Chen/EER notation.

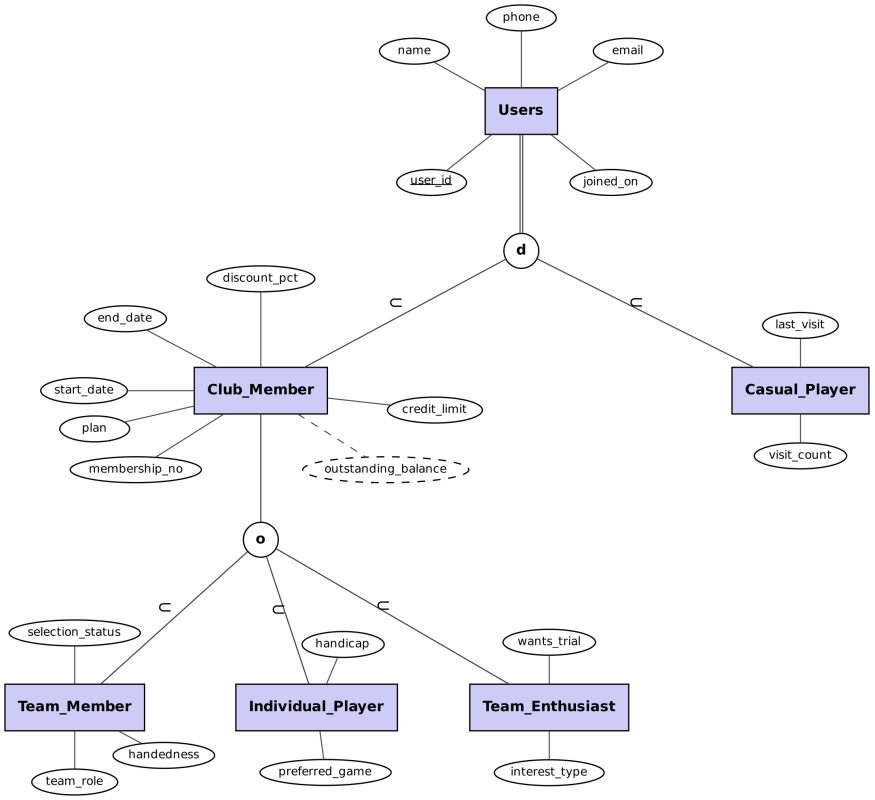



**Figure 1:** EER diagram of the Users specialization (Level 1: disjoint *d*, total; Level 2: overlapping *o*, partial).

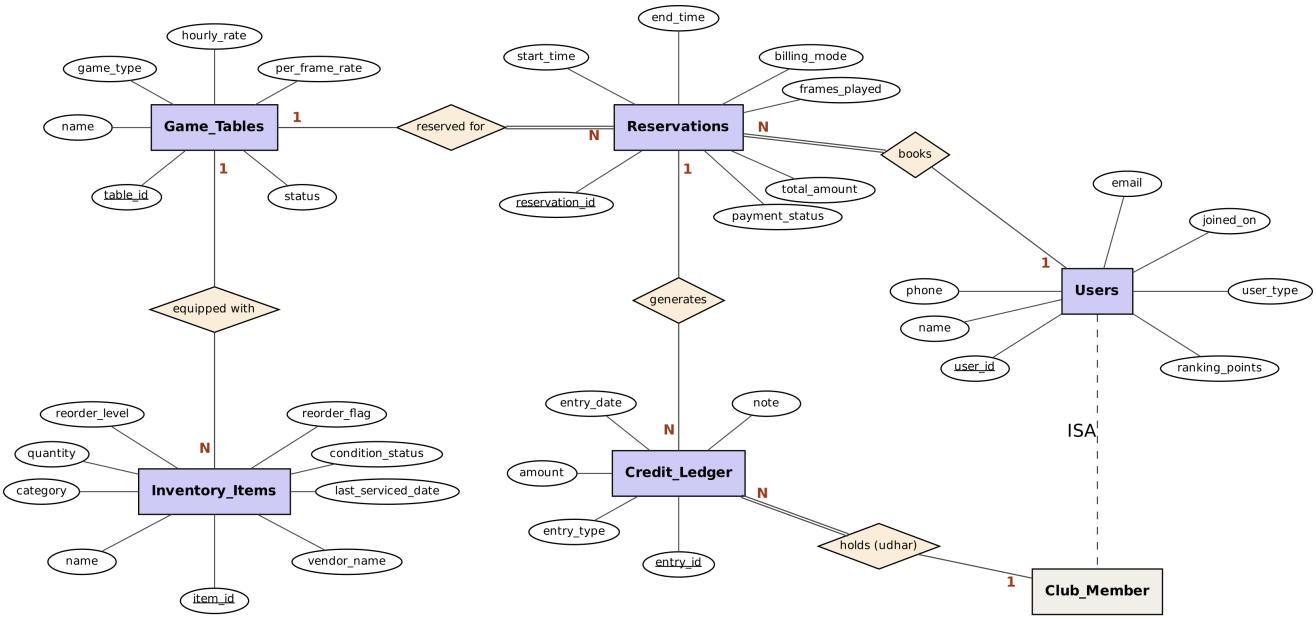



**Figure 2:** Conceptual ER diagram: Club operations, reservations, credit ledger and inventory.

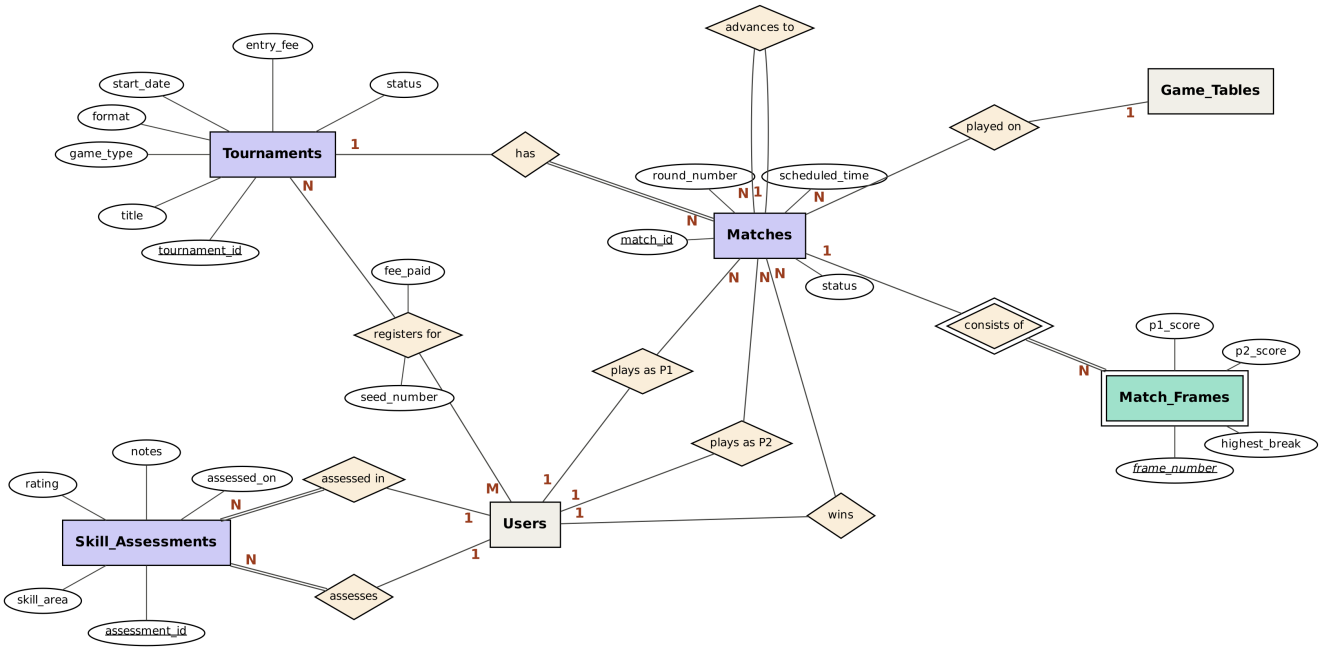



**Figure 3:** Conceptual ER diagram: Competition bracket tree, weak entity frames and coaching assessments.

| Relationship | Entities | Card. | Participation and Mapping Constraints |
|---|---|---|---|
| reserved for | Game_Tables, Reservations | 1:N | Reservations total; Game_Tables partial. Table assigned to reservation. |
| books | Users, Reservations | 1:N | Reservations total; Users partial. Identifies booking owner. |
| generates | Reservations, Credit_Ledger | 1:N | Reservations partial; Credit_Ledger partial. Generates booking charges. |
| holds (udhar) | Club_Member, Credit_Ledger | 1:N | Credit_Ledger total; Club_Member partial. Credit facility restricted to members. |
| equipped with | Game_Tables, Inventory_Items | 1:N | Inventory_Items partial; Game_Tables partial. Equipment assigned to tables. |
| has | Tournaments, Matches | 1:N | Matches total; Tournaments partial. Fixtures belonging to a tournament. |
| advances to | Matches, Matches (recursive) | 1:N | Both partial. Bracket tree: winner moves to the next-round match. |
| played on | Game_Tables, Matches | 1:N | Matches partial; Game_Tables partial. Table allocated for fixture. |
| consists of | Matches, Match_Frames | 1:N | Match_Frames total (weak entity, partial key frame_number). |
| registers for | Tournaments, Users | M:N | Maps to Tournament_Registrations; carries seed_number, fee_paid. |
| plays as P1 / P2 | Matches, Users | 1:N each | Users partial; Matches partial (slots NULL until feeder matches complete). |
| wins | Matches, Users | 1:N | Matches partial; Users partial. Stores match victor. |
| assessed in / assesses | Skill_Assessments, Users | 1:N each | Skill_Assessments total in both (player evaluated and coach assessing). |

**Table 2:** Relationships, cardinality, and participation constraints.

**EER mapping notes:** Level 1 specialization is disjoint and total; mapped with discriminator `user_type` in `Users`. `Casual_Player` has no specialized attributes and requires no separate relation. `Club_Member` is a distinct relation (PK = FK to `Users`). Level 2 specialization is overlapping and partial, mapped inside `Club_Member` via boolean flags (`is_team_member`, `is_individual_player`, `is_enthusiast`) guarding nullable attributes.

# 4) Relational Schema (11 Relations)

**Notation:** underline = primary key; *italic (>Table)* = foreign key; [U] = UNIQUE; [N] = nullable; everything else is NOT NULL.

| Relation | Attributes, Keys and Constraints |
|---|---|
| Users | user_id INT, name VARCHAR(100), phone VARCHAR(15) [U], email VARCHAR(120) [U], joined_on DATE, user_type ENUM, ranking_points INT |
| Club_Member | *user_id(>Users)* INT, membership_no VARCHAR(20) [U], plan ENUM, start_date DATE, end_date DATE, discount_pct DECIMAL(5,2), credit_limit DECIMAL(10,2), is_team_member BOOL, team_role ENUM [N], selection_status ENUM [N], handedness ENUM [N], is_individual_player BOOL, handicap INT [N], preferred_game ENUM [N], is_enthusiast BOOL, interest_type ENUM [N], wants_trial BOOL [N] |
| Game_Tables | table_id INT, name VARCHAR(50) [U], game_type ENUM, hourly_rate DECIMAL(8,2), per_frame_rate DECIMAL(8,2), status ENUM |
| Reservations | reservation_id INT, *user_id(>Users)* INT, *table_id(>Game_Tables)* INT, start_time DATETIME, end_time DATETIME, billing_mode ENUM, frames_played INT, total_amount DECIMAL(10,2), payment_status ENUM |
| Credit_Ledger | entry_id INT, *user_id(>Club_Member)* INT, *reservation_id(>Reservations)* INT [N], entry_type ENUM, amount DECIMAL(10,2), entry_date DATETIME, note VARCHAR(200) [N] |
| Tournaments | tournament_id INT, title VARCHAR(120), game_type ENUM, format ENUM, start_date DATE, entry_fee DECIMAL(8,2), status ENUM |
| Tournament_ Registrations | registration_id INT, *tournament_id(>Tournaments)* INT, *user_id(>Users)* INT, seed_number INT [N], fee_paid BOOL |
| Matches | match_id INT, *tournament_id(>Tournaments)* INT, round_number INT, *player1_id(>Users)* INT [N], *player2_id(>Users)* INT [N], *winner_id(>Users)* INT [N], *next_match_id(>Matches)* INT [N], *table_id(>Game_Tables)* INT [N], scheduled_time DATETIME [N], status ENUM |
| Match_Frames | *match_id(>Matches)*, frame_number INT, p1_score INT, p2_score INT, highest_break INT |
| Skill_Assessments | assessment_id INT, *player_id(>Users)* INT, *assessed_by(>Users)* INT, skill_area ENUM, rating TINYINT, notes VARCHAR(300) [N], assessed_on DATE |
| Inventory_Items | item_id INT, name VARCHAR(100), category ENUM, quantity INT, reorder_level INT, reorder_flag BOOL, *table_id(>Game_Tables)* INT [N], condition_status ENUM, last_serviced_date DATE [N], vendor_name VARCHAR(100) |

**Table 3:** Relational schema with primary keys, foreign key references, and nullability.

**ENUM domains:** user_type {Member, Casual}; plan {Monthly, Quarterly, Annual}; team_role {Captain, Vice-Captain, Player}; selection_status {Probable, Selected, Reserve, Dropped}; handedness {Right, Left}; game_type, preferred_game {Snooker, Pool, Billiards}; table status {Available, Maintenance, Retired}; billing_mode {Hourly, PerFrame}; payment_status {Pending, Paid, OnCredit, Cancelled}; entry_type {Booking charge, Canteen charge, Fine, Payment}; interest_type {Supporter, Aspiring Player, Volunteer}; format {Knockout, League}; tournament status {Draft, Registration, Ongoing, Completed, Cancelled}; match status {Scheduled, InProgress, Completed, Cancelled}; skill_area {Long potting, Safety play, Break building, Positional play, Cue ball control, Pressure handling}; category {Cue, Chalk, Cloth, Balls, Consumable, Other}; condition_status {Good, Needs Service, Damaged, Retired}.

# 5) Integrity Constraints Enforced by the DBMS

| Type | Constraint |
|---|---|
| Entity | Surrogate PKs; PK = FK on Club_Member; composite PK on Match_Frames. |
| Referential | Credit_Ledger.user_id → Club_Member (casual players cannot hold credit); Matches.next_match_id → Matches; player1_id, player2_id, winner_id → Users; Reservations.table_id → Game_Tables (RESTRICT); Skill_Assessments.player_id, assessed_by → Users. CASCADE on weak entities (Match_Frames), RESTRICT on financial/match records. |
| Domain / CHECK | end_time > start_time; quantity, reorder_level, ranking_points ≥ 0; hourly_rate, per_frame_rate, amount > 0; rating BETWEEN 1 AND 10; discount_pct BETWEEN 0 AND 100; end_date > start_date; player1_id ≠ player2_id; winner_id is NULL or one of the two players; p1_score, p2_score ≥ 0 and p1_score ≠ p2_score (no drawn frames); Club_Member: team_role, selection_status, handedness NOT NULL iff is_team_member (likewise for the other two flags); ENUMs for all status domains. |
| UNIQUE | Users(phone), Users(email), Club_Member(membership_no), Game_Tables(name), Tournament_Registrations (tournament_id, user_id), Tournament_Registrations (tournament_id, seed_number). |
| Triggers | Double booking: BEFORE INSERT/UPDATE on Reservations rejects an overlap on the same table (existing.start < new.end AND existing.end > new.start; cancelled bookings ignored). Casual upfront: a casual player's reservation must be Paid; OnCredit requires a club member. Credit limit: a charge is rejected if the balance would exceed credit_limit; a payment may not exceed the balance. Specialization: a Club_Member row requires user_type = Member; team roles require an active membership. Tournament: registration only while status is Registration; match players must be registered; next_match_id must belong to the same tournament. Coaching: only team members or individual players can be assessed. Inventory: reorder_flag is set when quantity < reorder_level. |
| Transactional | sp_complete_match: START TRANSACTION, insert frames, derive the winner, set winner_id, place the winner in the next match, add ranking points, COMMIT (ROLLBACK on any error). |

**Table 4:** Integrity constraints, domain validations, triggers, and transactional rules.

## 6) Normalization and Design Justification

| Relation | Non-Trivial Functional Dependencies | Normal Form |
|---|---|---|
| Users, Game_Tables, Tournaments | Candidate keys determine all non-key attributes; no transitive dependencies. | BCNF |
| Club_Member | user_id → all; membership_no → all; candidate keys determine all columns. | BCNF |
| Match_Frames | (match_id, frame_number) → p1_score, p2_score, highest_break (atomic row per frame). | BCNF (from 1NF) |
| Tournament_Registrations | (tournament_id, user_id) → seed_number, fee_paid. FDs on tournament_id factored out. | BCNF (from 2NF) |
| Reservations | reservation_id → all. Table rates moved to Game_Tables, customer details to Users. | BCNF (from 3NF) |

**Table 5:** Functional dependencies and normal form evaluation.

**Deliberate trade-offs and controlled redundancy:**

- **Snapshot total_amount:** Reservations.total_amount is stored as a snapshot to prevent historical billing corruption when rates change.
- **Materialized ranking_points:** Users.ranking_points is stored directly for fast leaderboard queries, updated atomically via `sp_complete_match`.
- **Derived outstanding_balance:** Outstanding credit is computed dynamically by a view (`v_member_balance`) from Credit_Ledger entries to prevent update anomalies.

## 7) Application Design

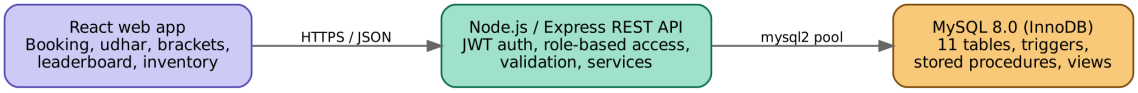



**Figure 4:** Three-tier architecture.

| # | Workflow | Demonstration and Integrity Check |
|---|---|---|
| W1 | Match completion | Scorer submits frames; `sp_complete_match` atomically records frames, sets winner, advances bracket, updates ranking points. |
| W2 | Leaderboard & reports | Dashboards with `RANK()`, win streaks (window CTEs), head-to-head records, peak table utilization, weakest skills. |
| W3 | Booking & credit | Staff reserves table; triggers prevent overlaps and casual credit; validates credit limits and enforces balance reductions. |
| W4 | Inventory & coaching | Automatic reorder flags on low stock; logs servicing dates; captain records skill assessments for team selection. |

**Table 6:** Planned end-to-end workflows.

## 8) Progress and Team Contributions

| Member | ID | Contribution |
|---|---|---|
| Priyansh Thadhani | 2024A3PS0356H | Core database design: ER/EER modeling, relational mapping, schema definition, and BCNF normalization. |
| Goru Nikhil | 2024A3PS0357H | Constraints design: Triggers, check conditions, foreign key cascading rules, and procedural match transaction. |
| Ananya Avunooru | 2024A3PS0367H | Application architecture: Express REST API design, JWT authentication, and MySQL connection pooling. |
| Manya Bhateja | 2024A3PS0378H | Reporting and views: Leaderboard views, recursive CTE bracket queries, and synthetic data generation script. |
| Ujjwal Chandak | 2024A3PS0379H | Frontend design: React UI for booking flow, credit ledger viewer, and documentation synthesis. |

**Table 7:** Team contributions up to mid-semester submission.